

EXPERIMENT 1: Python Program to Implement List Operations**Aim:**

To write a Python program using Anaconda (Spyder/Jupyter) that demonstrates the fundamental operations that can be performed on a list data structure, such as creation, accessing, updating, adding, removing, reversing, sorting, and concatenating elements.

Description:

- A list is an ordered, mutable (changeable) collection of items.
- Lists can hold heterogeneous data types (integers, strings, floats, objects) in the same list.
- Lists allow duplicate elements.
- Lists are dynamic in size, elements can be added or removed at runtime.
- Elements are accessed using a zero-based index, and negative indexing is supported.
- Lists support built-in methods like `append()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()`, etc.

Algorithm:

1. Start.
2. Create a list with a set of initial elements.
3. Access elements using positive and negative indices.
4. Update an element at a specific index with a new value.
5. Add a new element to the list using `append()`.
6. Remove a specified element using `remove()`.
7. Reverse the order of the list using `reverse()`.
8. Sort the list in ascending order using `sort()`.
9. Find the length of the list using `len()`.
10. Concatenate the list with another list using the `+` operator.
11. Repeat the list contents using the `*` operator.
12. Clear all elements from the list using `clear()`.
13. Display the result after every operation.
14. Stop.

Program (Anaconda / Python):

```
my_list = [10, 20, 30, 40, 50]
print("Original List:", my_list)
```

```
# Creating a list
list1 = [1, 2, 3]
```

```
print("Created List:", list1)

# Accessing elements
print("First element:", my_list[0])
print("Last element:", my_list[-1])

# Updating an element
my_list[2] = 300
print("List after update:", my_list)

# Adding elements
my_list.append(60)
print("List after append:", my_list)

# Removing an element
my_list.remove(40)
print("List after removing 40:", my_list)

# Reversing a list
my_list.reverse()
print("Reversed List:", my_list)

# Sorting a list
my_list.sort()
print("Sorted List:", my_list)

# Length of list
print("Length of List:", len(my_list))

# Concatenating two lists
list2 = [70, 80, 90]
concat_list = my_list + list2
print("Concatenated List:", concat_list)

# Repeating a list
print("Repeated List:", list1 * 2)

# Clearing a list
list1.clear()
print("List after clear:", list1)
```

Output:

Original List: [10, 20, 30, 40, 50]

Created List: [1, 2, 3]

First element: 10

Last element: 50

List after update: [10, 20, 300, 40, 50]

List after append: [10, 20, 300, 40, 50, 60]

List after removing 40: [10, 20, 300, 50, 60]

Reversed List: [60, 50, 300, 20, 10]

Sorted List: [10, 20, 50, 60, 300]

Length of List: 5

Concatenated List: [10, 20, 50, 60, 300, 70, 80, 90]

Repeated List: [1, 2, 3, 1, 2, 3]

List after clear: []

EXPERIMENT 2: Implementation of Nested List, Length, Concatenation, Membership, Iteration, Indexing and Slicing Functions.

Aim:

To write a Python program that demonstrates nested lists and the core list functions/operators: length, concatenation, membership testing, iteration, indexing and slicing.

Description:

- A nested list is a list that contains other lists as its elements, allowing multi-dimensional data representation (e.g., matrices, tables).
- Length (`len()`) returns the number of top-level elements in a list.
- Concatenation (`+`) joins two or more lists into a single new list without altering the originals.
- Membership (`in` / `not in`) checks whether a value exists within a list, returning a Boolean.
- Iteration allows traversal of each element using a for loop.
- Indexing accesses a single element using its position (positive from the start, negative from the end).
- Slicing (`start:stop:step`) extracts a sub-list/range of elements without modifying the original list.

Algorithm:

1. Start.
2. Create a nested list containing lists of different data types.
3. Access an inner element using double indexing, e.g. `list[i][j]`.
4. Compute the length of the outer list using `len()`.
5. Create two simple lists and concatenate them using `+`.
6. Check membership of a value in the concatenated list using `in`.
7. Iterate through the list using a for loop and print each element.
8. Demonstrate indexing using positive and negative indices.
9. Demonstrate slicing using `[start:stop]`, `[:stop]`, `::step`, and `::-1`.
10. Display results for each demonstration.
11. Stop.

Program (Anaconda / Python)

```
# Nested List
nested_list = [[1, 2, 3], ["a", "b", "c"], [True, False]]
print("Nested List:", nested_list)
print("Element at nested_list[1][2]:", nested_list[1][2])

# Length
print("\nLength of nested list:", len(nested_list))
```

```
# Concatenation
list_a = [1, 2, 3]
list_b = [4, 5, 6]
concatenated = list_a + list_b
print("\nConcatenation of", list_a, "and", list_b, "=", concatenated)

# Membership
print("\nIs 5 in concatenated list?", 5 in concatenated)
print("Is 10 in concatenated list?", 10 in concatenated)

# Iteration
print("\nIterating through concatenated list:")
for item in concatenated:
    print("Item:", item)

# Indexing
sample = [10, 20, 30, 40, 50]
print("\nIndexing:")
print("sample[0] =", sample[0])
print("sample[-1] =", sample[-1])

# Slicing
print("\nSlicing:")
print("sample[1:4] =", sample[1:4])
print("sample[:3] =", sample[:3])
print("sample[::2] =", sample[::2])
print("sample[::-1] =", sample[::-1])
```

Output:

Nested List: [[1, 2, 3], ['a', 'b', 'c'], [True, False]]

Element at nested_list[1][2]: c

Length of nested list: 3

Concatenation of [1, 2, 3] and [4, 5, 6] = [1, 2, 3, 4, 5, 6]

Is 5 in concatenated list? True

Is 10 in concatenated list? False

Iterating through concatenated list:

Item: 1

Item: 2

Item: 3

Item: 4

Item: 5

Item: 6

Indexing:

sample[0] = 10

sample[-1] = 50

Slicing:

sample[1:4] = [20, 30, 40]

sample[:3] = [10, 20, 30]

sample[::2] = [10, 30, 50]

sample[::-1] = [50, 40, 30, 20, 10]

EXPERIMENT 3: Implementation of List Methods — Add, Append, Extend & Delete**Aim:**

To write a Python program to demonstrate the list methods used for adding elements (insert, append, extend) and deleting elements (remove, pop, del, clear).

Description:

- ``append()`` adds a single element to the end of the list.
- ``insert(index, element)`` adds an element at a specific position ("add"), shifting subsequent elements to the right.
- ``extend(iterable)`` appends every element of another iterable individually onto the end of the list (unlike `append`, which would add the whole iterable as one element).
- ``remove(value)`` deletes the first occurrence of a given value.
- ``pop(index)`` deletes and returns the element at the given index (last element by default).
- ``del`` statement deletes an element by index or the entire list.
- ``clear()`` empties the list completely, leaving `[]`.

Algorithm:

1. Start.
2. Create a list of fruit names.
3. Append a new fruit to the end of the list.
4. Insert a fruit at a specific index (Add operation).
5. Extend the list with another list of fruits.
6. Remove a fruit using `remove()` by value.
7. Delete a fruit using `pop()` by index and display the removed item.
8. Delete a fruit using the `del` statement.
9. Clear a copied list using `clear()` and show the original list remains unaffected.
10. Display the list after every operation.
11. Stop.

Program (Anaconda / Python):

```
fruits = ["apple", "banana", "cherry"]
print("Original List:", fruits)

# Append - adds single element at end
fruits.append("mango")
```

```
print("\nAfter append('mango'):", fruits)

# Insert (Add at a specific position)
fruits.insert(1, "orange")
print("\nAfter insert(1, 'orange'):", fruits)

# Extend - adds elements of another iterable
more_fruits = ["grape", "kiwi"]
fruits.extend(more_fruits)
print("\nAfter extend(['grape', 'kiwi']):", fruits)

# Delete using remove() - removes by value
fruits.remove("banana")
print("\nAfter remove('banana'):", fruits)

# Delete using pop() - removes by index, returns removed item
popped_item = fruits.pop(0)
print("\nAfter pop(0), removed item:", popped_item, "-> List:", fruits)

# Delete using del statement
del fruits[2]
print("\nAfter del fruits[2]:", fruits)

# Delete using clear()
temp_list = fruits.copy()
temp_list.clear()
print("\nAfter clear() on a copy:", temp_list)
print("Original list unaffected:", fruits)
```

Output:

Original List: ['apple', 'banana', 'cherry']

After append('mango'): ['apple', 'banana', 'cherry', 'mango']

After insert(1, 'orange'): ['apple', 'orange', 'banana', 'cherry', 'mango']

After extend(['grape', 'kiwi']): ['apple', 'orange', 'banana', 'cherry', 'mango', 'grape', 'kiwi']

After remove('banana'): ['apple', 'orange', 'cherry', 'mango', 'grape', 'kiwi']

After pop(0), removed item: apple -> List: ['orange', 'cherry', 'mango', 'grape', 'kiwi']

After del fruits[2]: ['orange', 'cherry', 'grape', 'kiwi']

After clear() on a copy: []

Original list unaffected: ['orange', 'cherry', 'grape', 'kiwi']

EXPERIMENT 4: Program to Eliminate Punctuations in a Given String**Aim:**

To write a Python program that reads a string and removes all punctuation marks from it.

Description:

- Python's built-in string module provides a constant string. punctuation, containing all standard punctuation characters: `!"#\$%&'()\*+,-./:;<=>?@[\\]^\_`{|}~``.
- Strings in Python are immutable, so punctuation removal requires building a new string rather than modifying the original.
- str.translate() combined with str.maketrans() gives an efficient, single-pass way to strip unwanted characters compared to repeated concatenation.
- This kind of pre-processing is a common text-cleaning / NLP pre-processing step used before tokenization.

Algorithm:

1. Start.
2. Import the string module to access the set of punctuation characters.
3. Take an input string containing punctuation marks.
4. Method 1 (Loop-based): Traverse each character of the string; if the character is not present in string.punctuation, append it to a result string.
5. Method 2 (Translate-based): Build a translation table using str.maketrans("", "", string.punctuation) and apply str.translate() to remove punctuation in one step.
6. Display the cleaned string produced by both methods.
7. Stop.

Program (Anaconda / Python):

```
import string

def remove_punctuation(text):
    result = ""
    for char in text:
        if char not in string.punctuation:
            result += char
    return result

input_string = "Hello!!! Are you, ready? Let's start... the A.I. Lab-Program, now!"
```

```
print("Original String:", input_string)

cleaned_string = remove_punctuation(input_string)
print("String after removing punctuation:", cleaned_string)

# Alternative method using str.translate()
translator = str.maketrans("", "", string.punctuation)
cleaned_string2 = input_string.translate(translator)
print("\nUsing translate() method:", cleaned_string2)
```

Output:

Original String: Hello!!! Are you, ready? Let's start... the A.I. Lab-Program, now!

String after removing punctuation: Hello Are you ready Lets start the AI LabProgram now

Using translate() method: Hello Are you ready Lets start the AI LabProgram now

EXPERIMENT 5: Sorting a Sentence in Alphabetical Order

Aim:

To write a Python program that accepts a sentence and arranges its words in alphabetical order.

Description:

- A sentence is first tokenized into individual words using `split()`, which by default splits on whitespace.
- Python's `sorted()` function sorts an iterable and returns a new list, leaving the original unchanged, while `list.sort()` sorts the list in place.
- String comparison in Python is case-sensitive (uppercase letters sort before lowercase ASCII-wise), so a key function such as `str.lower` is used to perform a case-insensitive sort.
- The `join()` method is used to combine the sorted words back into a single sentence string.

Algorithm:

1. Start.
2. Accept/define a sentence.
3. Split the sentence into a list of words using `split()`.
4. Convert each word to lowercase and strip trailing punctuation for a fair alphabetical comparison.
5. Sort the list of words alphabetically using `sorted()`.
6. Join the sorted words back into a single string using `" ".join()`.
7. (Optional) Sort while preserving the original case of each word using a key argument.
8. Display the sorted sentence.
9. Stop.

Program (Anaconda / Python):

```
def sort_sentence(sentence):  
    words = sentence.split()  
    words_lower = [word.strip(".,!?").lower() for word in words]  
    sorted_words = sorted(words_lower)  
    return " ".join(sorted_words)  
  
sentence = "Artificial Intelligence is the future of modern technology"  
print("Original Sentence:", sentence)  
sorted_sentence = sort_sentence(sentence)  
print("Alphabetically Sorted Sentence:", sorted_sentence)
```

Sorting words but preserving original case

```
words = sentence.split()
```

```
sorted_words_case_preserved = sorted(words, key=lambda w: w.lower())
```

```
print("\nSorted (original case preserved):", " ".join(sorted_words_case_preserved))
```

Output:

Original Sentence: Artificial Intelligence is the future of modern technology

Alphabetically Sorted Sentence: artificial future intelligence is modern of technology the

Sorted (original case preserved): Artificial future Intelligence is modern of technology the

EXPERIMENT 6: Implementation of Breadth First Search (BFS) Traversal Technique**Aim:**

To implement the Breadth First Search (BFS) graph traversal algorithm in Python and display the order in which nodes are visited.

Description:

- BFS is an uninformed / blind search technique that explores a graph level by level (breadth-wise) before going deeper.
- It uses a queue (FIFO) data structure to keep track of nodes to be explored next.
- BFS guarantees the shortest path (in terms of number of edges) from the source to any other node in an unweighted graph.
- It is complete (it will find a solution if one exists) but can be memory-intensive, since it must store all nodes at the current level.
- Time complexity is $O(V + E)$, where V is the number of vertices and E is the number of edges.

Algorithm:

1. Start.
2. Represent the graph as an adjacency list (dictionary of node $\rightarrow$ list of neighbours).
3. Initialize an empty visited set and a queue containing only the start node.
4. Mark the start node as visited and enqueue it.
5. While the queue is not empty:
 - a. Dequeue the front node and add it to the traversal order.
 - b. For every unvisited neighbour of this node, mark it visited and enqueue it.
6. Repeat step 5 until the queue is empty.
7. Return/display the traversal order.
8. Stop.

Program (Anaconda / Python):

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B', 'F'],  
    'F': ['C', 'E']}
```

```
}  
  
def bfs(graph, start):  
    visited = set()  
    queue = deque([start])  
    visited.add(start)  
    traversal_order = []  
    while queue:  
        node = queue.popleft()  
        traversal_order.append(node)  
        for neighbour in graph[node]:  
            if neighbour not in visited:  
                visited.add(neighbour)  
                queue.append(neighbour)  
    return traversal_order  
  
print("Graph (Adjacency List):")  
for node, neighbours in graph.items():  
    print(f" {node} -> {neighbours}")  
start_node = 'A'  
result = bfs(graph, start_node)  
print(f"\nBFS Traversal starting from '{start_node}':")  
print(" -> ".join(result))
```

Output:

Graph (Adjacency List):

A -> ['B', 'C']

B -> ['A', 'D', 'E']

C -> ['A', 'F']

D -> ['B']

E -> ['B', 'F']

F -> ['C', 'E']

BFS Traversal starting from 'A':

A -> B -> C -> D -> E -> F

EXPERIMENT 7: Depth First Search (DFS) Traversal Technique Implementation

Aim:

To implement the Depth First Search (DFS) graph traversal algorithm in Python, using both a recursive and an iterative (stack-based) approach.

Description:

- DFS is an uninformed search technique that explores as far as possible along each branch before backtracking.
- It uses a stack (LIFO) data structure — either implicitly via recursion (the call stack) or explicitly using a list/stack.
- DFS is not guaranteed to find the shortest path, but it uses less memory than BFS in many cases since it doesn't need to store an entire level of nodes at once.
- DFS can get stuck in infinite loops on graphs with cycles unless a visited set is maintained.
- Time complexity is $O(V + E)$.

Algorithm:

1. Start.
2. Represent the graph as an adjacency list.
3. Recursive DFS:
 - a. Mark the current node as visited and add it to the traversal order.
 - b. For each unvisited neighbour of the current node, recursively call DFS on it.
4. Iterative DFS (using an explicit stack):
 - a. Push the start node onto the stack.
 - b. While the stack is not empty, pop a node; if it hasn't been visited, mark it visited, add it to the traversal order, and push its unvisited neighbours (in reverse order, to mimic recursive order).
5. Display the traversal order obtained from both methods.
6. Stop.

Program (Anaconda / Python):

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B', 'F'],
```

```
'F': ['C', 'E']
}

def dfs(graph, start, visited=None, traversal_order=None):
    if visited is None:
        visited = set()
    if traversal_order is None:
        traversal_order = []

    visited.add(start)
    traversal_order.append(start)

    for neighbour in graph[start]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited, traversal_order)

    return traversal_order

print("Graph (Adjacency List):")
for node, neighbours in graph.items():
    print(f" {node} -> {neighbours}")

start_node = 'A'
result = dfs(graph, start_node)
print(f"\nDFS Traversal starting from '{start_node}':")
print(" -> ".join(result))

# Iterative version using a stack
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    traversal_order = []

    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            traversal_order.append(node)
            for neighbour in reversed(graph[node]):
```

```
        if neighbour not in visited:
            stack.append(neighbour)
    return traversal_order
```

```
print("\nDFS (Iterative using Stack) starting from 'A':")
print(" -> ".join(dfs_iterative(graph, 'A')))
```

Output:

Graph (Adjacency List):

```
A -> ['B', 'C']
B -> ['A', 'D', 'E']
C -> ['A', 'F']
D -> ['B']
E -> ['B', 'F']
F -> ['C', 'E']
```

DFS Traversal starting from 'A':

```
A -> B -> D -> E -> F -> C
```

DFS (Iterative using Stack) starting from 'A':

```
A -> B -> D -> E -> F -> C
```

EXPERIMENT 8: Implementation of Water Jug Problem Using Heuristic Method**Aim:**

To solve the classic Water Jug Problem — given two jugs of fixed capacities and no measuring marks, find a sequence of pour/fill/empty operations to measure out a target quantity of water — using a heuristic, state-space search method implemented in Python.

Problem Statement:

Two jugs with capacities of 4 litres and 3 litres are available, with no other measuring device. Determine a sequence of operations to obtain exactly 2 litres in the 4-litre jug, starting from both jugs empty.

Description:

- The problem is modeled as a state-space search, where each state is a pair (x, y) representing the current water level in Jug 1 and Jug 2.
- The available operators/production rules at each state are: fill Jug 1, fill Jug 2, empty Jug 1, empty Jug 2, pour Jug 1 into Jug 2, and pour Jug 2 into Jug 1.
- A heuristic/systematic search (here, BFS over the state space) is used to guarantee the shortest sequence of moves to reach the goal state.
- A visited set prevents revisiting the same state and avoids infinite loops.
- This is a classic example of representing a real-world puzzle as a graph search problem in AI.

Algorithm:

1. Start.
2. Define jug capacities cap1, cap2, and the target quantity goal.
3. Initialize the start state as (0, 0) (both jugs empty) and add it to visited.
4. Initialize a queue with the start state and its path so far.
5. While the queue is not empty:
 - a. Dequeue the current state (x, y) and its path.
 - b. If $x == \text{goal}$ or $y == \text{goal}$, return the path as the solution.
 - c. Generate all possible next states by applying the six operators (fill Jug1, fill Jug2, empty Jug1, empty Jug2, pour Jug1→Jug2, pour Jug2→Jug1).
 - d. For every new, unvisited state, mark it visited and enqueue it along with the updated path.
6. If the queue becomes empty without reaching the goal, report "No solution".
7. Display the resulting sequence of states (Jug1, Jug2) leading to the goal.
8. Stop.

Program (Anaconda / Python):

```
from collections import deque  
  
def water_jug_bfs(cap1, cap2, goal):
```

```
start = (0, 0)
visited = set([start])
queue = deque([(start, [start])])
while queue:
    (x, y), path = queue.popleft()
    if x == goal or y == goal:
        return path
    possible_next_states = [
        (cap1, y),          # Fill Jug1 completely
        (x, cap2),          # Fill Jug2 completely
        (0, y),             # Empty Jug1
        (x, 0),             # Empty Jug2
        (x - min(x, cap2 - y), y + min(x, cap2 - y)), # Pour Jug1 -> Jug2
        (x + min(y, cap1 - x), y - min(y, cap1 - x)), # Pour Jug2 -> Jug1
    ]
    for state in possible_next_states:
        if state not in visited:
            visited.add(state)
            queue.append((state, path + [state]))
    return None
cap1, cap2, goal = 4, 3, 2
solution = water_jug_bfs(cap1, cap2, goal)
print(f"Jug1 Capacity = {cap1} litres, Jug2 Capacity = {cap2} litres, Goal = {goal} litres")
print("\nSequence of states (Jug1, Jug2) to reach the goal:")
if solution:
    for step, state in enumerate(solution):
        print(f"Step {step}: {state}")
else:
    print("No solution found.")
```

Output:

Jug1 Capacity = 4 litres, Jug2 Capacity = 3 litres, Goal = 2 litres

Sequence of states (Jug1, Jug2) to reach the goal:

Step 0: (0, 0)

Step 1: (0, 3)

Step 2: (3, 0)

Step 3: (3, 3)

Step 4: (4, 2)

Explanation of Output

- Step 0: Both jugs empty (0, 0).
- Step 1: Jug 2 (3L) filled completely $\rightarrow$ (0, 3).
- Step 2: Jug 2's contents poured into Jug 1 $\rightarrow$ (3, 0).
- Step 3: Jug 2 filled again $\rightarrow$ (3, 3).
- Step 4: Jug 2 poured into Jug 1 until Jug 1 is full (capacity 4), leaving exactly 2 litres in Jug 2 $\rightarrow$ (4, 2) — goal reached.

EXPERIMENT 9: Implementation of Travelling Salesman Problem (TSP) Using Python**Aim:**

To implement the Travelling Salesman Problem (TSP) in Python — finding the shortest possible route that visits every city exactly once and returns to the starting city.

Description:

- TSP is a classic NP-Hard combinatorial optimization problem in AI and operations research.
- The brute-force approach used here evaluates every possible permutation of the cities (excluding the fixed start city) and computes the total travel cost for each route.
- The graph is represented as a weighted, complete graph using an adjacency matrix (here, a dictionary of dictionaries) storing distances between every pair of cities.
- The brute-force method guarantees the globally optimal solution but has a factorial time complexity of $O((n-1)!)$, making it impractical for a large number of cities — hence in practice, heuristic methods (nearest neighbour, genetic algorithms, branch-and-bound) are used for bigger instances.

Algorithm:

1. Start.
2. Represent the cities and the distances between every pair of cities using a distance matrix / dictionary.
3. Fix a starting city and generate all possible permutations of the remaining cities using `itertools.permutations`.
4. For each permutation:
 - a. Compute the total cost of travelling from the start city through the permutation and back to the start city.
 - b. If this cost is lower than the current minimum cost, update the minimum cost and store this route as the best route.
5. After evaluating all permutations, output the best (optimal) route and its total cost.
6. Stop.

Program (Anaconda / Python):

```
from itertools import permutations

def travelling_salesman(graph, start):
    vertices = list(graph.keys())
    vertices.remove(start)
```

```
min_path_cost = float('inf')
best_route = None

for perm in permutations(vertices):
    current_cost = 0
    current_city = start
    route = [start]
    for city in perm:
        current_cost += graph[current_city][city]
        current_city = city
        route.append(city)
    current_cost += graph[current_city][start]
    route.append(start)
    if current_cost < min_path_cost:
        min_path_cost = current_cost
        best_route = route
return best_route, min_path_cost

# Distance matrix represented as a dictionary of dictionaries
graph = {
    'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},
    'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},
    'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},
    'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}
}

print("Distance Matrix:")
for city, dist in graph.items():
    print(f" {city}: {dist}")

best_route, cost = travelling_salesman(graph, 'A')
print("\nOptimal Route:", " -> ".join(best_route))
print("Minimum Travel Cost:", cost)
```

Output

Distance Matrix:

A: {'A': 0, 'B': 10, 'C': 15, 'D': 20}

B: {'A': 10, 'B': 0, 'C': 35, 'D': 25}

C: {'A': 15, 'B': 35, 'C': 0, 'D': 30}

D: {'A': 20, 'B': 25, 'C': 30, 'D': 0}

Optimal Route: A -> B -> D -> C -> A

Minimum Travel Cost: 80

EXPERIMENT 10: Writing Simple Facts for the Statements and Querying Using Prolog.**Aim:**

To write simple facts and rules in Prolog for a set of given statements and to query the Prolog knowledge base to retrieve information.

Description of Prolog:

- Prolog (PROgramming in LOGic) is a declarative, logic-based programming language used widely in AI for knowledge representation and reasoning.
- A Prolog program consists of a set of facts (unconditionally true statements), rules (conditional statements built using facts and other rules), and queries (questions posed against the knowledge base).
- Prolog uses unification and backtracking to match queries against facts/rules and to explore all possible solutions.
- Predicates are structured as predicate_name(argument1, argument2, ...).
- The forall/2 and format/2 built-in predicates are commonly used to iterate over solutions and print formatted output.

Algorithm:

1. Start.
2. Define a set of facts representing relationships between entities (e.g., likes(john, mango)).
3. Define a rule (a conditional/derived statement) that combines two or more facts, e.g. common_liking/3 — true if two different people like the same fruit.
4. Load the Prolog file into the SWI-Prolog interpreter.
5. Pose simple queries directly against facts (e.g., "does John like mango?").
6. Use forall/2 to enumerate all facts satisfying a given predicate (e.g., all fruits John likes).
7. Use the derived rule to find and print all pairs of people who share a common liking.
8. Halt the program after printing all results.
9. Stop.

Program (Prolog — save as `facts\_query.pl`, run using SWI-Prolog: `swipl facts\_query.pl`)

```
% Facts  
likes(john, mango).  
likes(mary, apple).  
likes(john, apple).  
likes(peter, banana).  
likes(mary, mango).
```

```

fruit(mango).
fruit(apple).
fruit(banana).

% Rule: common_liking - two people who like the same fruit
common_liking(X, Y, F) :-
    likes(X, F),
    likes(Y, F),
    X \== Y.

:- initialization(main).

main :-
    write('--- Facts about likes ---'), nl,
    ( likes(john, mango) -> write('john likes mango: true') ; write('john likes mango: false') ), nl,
    ( likes(peter, apple) -> write('peter likes apple: true') ; write('peter likes apple: false') ), nl,
    nl,
    write('--- All fruits John likes ---'), nl,
    forall(likes(john, F), (write(F), nl)),
    nl,
    write('--- All people who like mango ---'), nl,
    forall(likes(P, mango), (write(P), nl)),
    nl,
    write('--- Pairs who like the same fruit ---'), nl,
    forall(common_liking(X, Y, F), format("~w and ~w both like ~w~n", [X, Y, F])),
    halt.

```

Output:

```

--- Facts about likes ---
john likes mango: true
peter likes apple: false
--- All fruits John likes ---
mango
apple
--- All people who like mango ---
john

```

mary

--- Pairs who like the same fruit ---

john and mary both like mango

mary and john both like apple

john and mary both like apple

mary and john both like mango

EXPERIMENT 11: Demonstration of Family Tree Using Prolog Using Conditional Statements**Aim:**

To represent a family tree using facts in Prolog and to derive relationships such as father, mother, grandparent, grandfather, grandmother, and siblings using conditional statements (rules).

Description:

- Facts (male/1, female/1, parent/2) represent the base, ground-truth family data.
- Rules are conditional statements of the form Head :- Body, meaning "Head is true IF Body is true". They allow Prolog to derive new knowledge rather than storing every relationship explicitly.
- Relationships such as grandparent/2 are built by composing simpler rules (e.g. two chained parent/2 facts), demonstrating how Prolog supports modular, reusable logic.
- The distinct/1 predicate is used to avoid duplicate results caused by backtracking over multiple valid unifications.
- This experiment illustrates Prolog's strength in relational knowledge representation and automatic inference, which is central to symbolic AI.

Algorithm:

1. Start.
2. Define base facts: male/1, female/1 for gender, and parent/2 for parent-child relationships.
3. Define the rule father(F, C): F is the father of C if F is a parent of C and F is male.
4. Define the rule mother(M, C): M is the mother of C if M is a parent of C and M is female.
5. Define the rule grandparent(GP, GC): GP is a grandparent of GC if GP is a parent of some P, and P is a parent of GC.
6. Define grandfather/2 and grandmother/2 by combining grandparent/2 with gender facts.
7. Define sibling(X, Y): X and Y are siblings if they share a common parent and are not the same person; refine into brother/2 and sister/2 using gender.
8. Query and print, using forall/2, all instances of each relationship (father, mother, grandfather, grandmother, sibling).
9. Halt.
10. Stop.

Program (Prolog — save as `family\_tree.pl`, run using SWI-Prolog: `swipl family\_tree.pl`)

```
% Facts
male(john).
male(peter).
male(paul).
male(michael).
```

```
female(susan).
female(mary).
female(alice).
female(linda).
```

```
parent(john, peter).
parent(john, alice).
parent(susan, peter).
parent(susan, alice).
parent(peter, paul).
parent(peter, linda).
parent(mary, paul).
parent(mary, linda).
parent(peter, michael).
parent(mary, michael).
```

```
% Rules (conditional statements)
father(F, C) :- parent(F, C), male(F).
mother(M, C) :- parent(M, C), female(M).
```

```
grandparent(GP, GC) :- parent(GP, P), parent(P, GC).
grandfather(GF, GC) :- grandparent(GF, GC), male(GF).
grandmother(GM, GC) :- grandparent(GM, GC), female(GM).
```

```
sibling(X, Y) :- parent(P, X), parent(P, Y), X \== Y.
brother(B, X) :- sibling(B, X), male(B).
sister(S, X) :- sibling(S, X), female(S).
```

```
:- initialization(main).
```

```
main :-
```

```
    write('--- Father Relationships ---'), nl,
    forall(father(F, C), format("~w is the father of ~w~n", [F, C])),
    nl,
    write('--- Mother Relationships ---'), nl,
    forall(mother(M, C), format("~w is the mother of ~w~n", [M, C])),
    nl,
    write('--- Grandfather Relationships ---'), nl,
    forall(distinct(grandfather(GF, GC)), format("~w is the grandfather of ~w~n", [GF, GC])),
    nl,
```

```
write('--- Grandmother Relationships ---'), nl,
forall(distinct(grandmother(GM, GC)), format("~w is the grandmother of ~w~n", [GM, GC])),
nl,
write('--- Sibling Relationships ---'), nl,
forall(distinct(sibling(X, Y)), format("~w and ~w are siblings~n", [X, Y])),
halt.
```

Output:

--- Father Relationships ---

john is the father of peter
john is the father of alice
peter is the father of paul
peter is the father of linda
peter is the father of michael

--- Mother Relationships ---

susan is the mother of peter
susan is the mother of alice
mary is the mother of paul
mary is the mother of linda
mary is the mother of michael

--- Grandfather Relationships ---

john is the grandfather of paul
john is the grandfather of linda
john is the grandfather of michael

--- Grandmother Relationships ---

susan is the grandmother of paul
susan is the grandmother of linda
susan is the grandmother of michael

--- Sibling Relationships ---

peter and alice are siblings
alice and peter are siblings
paul and linda are siblings
paul and michael are siblings
linda and paul are siblings
linda and michael are siblings
michael and paul are siblings
michael and linda are siblings

EXPERIMENT 12: CASE STUDY**12(a). Simple Chatroom Using Python****Aim:**

To design and implement a simple multi-user chatroom application in Python using socket programming and multi-threading, following the client-server model.

Description:

- Uses Python's built-in `socket` module for network communication over TCP/IP (AF_INET, SOCK_STREAM).
- Follows the client-server architecture: one server process manages all connections; multiple client processes connect independently.
- Uses multi-threading (threading module) so the server can handle several clients concurrently, and so each client can send and receive messages simultaneously without blocking.
- The server broadcasts every incoming message to all connected clients, giving the effect of a shared chatroom.
- Each client is asked for a nickname upon connecting, which is used to identify their messages to others.
- Demonstrates core Distributed AI / networked systems concepts: concurrency, message passing, and event-driven communication.

Algorithm:**Server side:**

1. Start.
2. Create a TCP socket, bind it to a host and port, and start listening for incoming connections.
3. Maintain two lists: clients (socket objects) and nicknames (their names).
4. In an infinite loop, accept new client connections.
5. For each new client: request its nickname, store the client and nickname, broadcast a "joined the chat" message to everyone, and confirm connection to the new client.
6. Spawn a new thread per client to continuously receive messages from that client and broadcast them to all other clients.
7. If a client disconnects, remove it from the lists and broadcast a "left the chat" message.
8. Stop (runs until manually terminated).

Client side:

1. Start.
2. Create a socket and connect to the server's host and port.

3. When prompted with 'NICK' by the server, send the chosen nickname.
4. Start one thread to continuously listen for and print incoming messages from the server.
5. Start another thread to continuously read user input and send it to the server.
6. Stop (runs until manually terminated).

Program (Anaconda / Python — combined server + client logic in one file for demonstration)

```
import socket
import threading

HOST = '127.0.0.1'
PORT = 55555

# ----- SERVER LOGIC -----
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()

    clients = []
    nicknames = []

    def broadcast(message):
        for client in clients:
            client.send(message)

    def handle_client(client):
        while True:
            try:
                message = client.recv(1024)
                broadcast(message)
            except:
                index = clients.index(client)
                clients.remove(client)
                client.close()
                nickname = nicknames[index]
                broadcast(f'{nickname} left the chat!'.encode('utf-8'))
                nicknames.remove(nickname)
                break
```

```
def receive():
    while True:
        client, address = server.accept()
        print(f'Connected with {str(address)}')

        client.send('NICK'.encode('utf-8'))
        nickname = client.recv(1024).decode('utf-8')
        nicknames.append(nickname)
        clients.append(client)

        print(f'Nickname of the client is {nickname}')
        broadcast(f'{nickname} joined the chat!'.encode('utf-8'))
        client.send('Connected to the server!'.encode('utf-8'))

        thread = threading.Thread(target=handle_client, args=(client,))
        thread.start()

print("Chat Server is listening...")
receive()

# ----- CLIENT LOGIC -----
def start_client(nickname):
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((HOST, PORT))

def receive_messages():
    while True:
        try:
            message = client.recv(1024).decode('utf-8')
            if message == 'NICK':
                client.send(nickname.encode('utf-8'))
            else:
                print(message)
        except:
            print("An error occurred!")
            client.close()
            break

def write_messages():
    while True:
```

```
message = f'{nickname}: {input("")}'  
client.send(message.encode('utf-8'))
```

```
receive_thread = threading.Thread(target=receive_messages)  
receive_thread.start()
```

```
write_thread = threading.Thread(target=write_messages)  
write_thread.start()
```

```
if __name__ == "__main__":  
    import sys  
    if len(sys.argv) > 1 and sys.argv[1] == 'server':  
        start_server()  
    elif len(sys.argv) > 2 and sys.argv[1] == 'client':  
        start_client(sys.argv[2])
```

How to run (2 or more terminals required, since it is a live networked application):

Terminal 1: python chatroom.py server

Terminal 2: python chatroom.py client John

Terminal 3: python chatroom.py client Mary

Output (sample chat session log, verified by running the server and two simultaneous clients)

[Server console]

Connected with ('127.0.0.1', 51410)

Nickname of the client is John

Connected with ('127.0.0.1', 51412)

Nickname of the client is Mary

[John's terminal]

Connected to the server!

Mary joined the chat!

John: Hi everyone!

Mary: Hello John!

[Mary's terminal]

Connected to the server!

Connected to the server!

John: Hi everyone!

Mary: Hello John!

12(b). Hospital Management System — Case Study Project in Python

Aim:

To design and implement a simple Hospital Management System in Python using Object-Oriented Programming (OOP) concepts, allowing patient records to be added, searched, updated, discharged, and displayed.

Description:

- Built using Object-Oriented Programming: a Patient class models each patient's data, and a HospitalManagementSystem class manages the overall collection of patients.
- Patient records are stored in a dictionary, keyed by a unique, auto-incrementing patient_id, giving O(1) average lookup time.
- Supports core CRUD-style operations: Create (admit), Read (search/display), Update (modify disease/doctor), and a soft Delete (discharge, which changes status rather than removing the record — preserving medical history).
- Encapsulation is used: each Patient object manages its own attributes and a custom __str__ method for clean, readable display.
- This case study demonstrates how AI/Python programming concepts (data structures + OOP) can be applied to build a realistic, menu-driven real-world management information system.

Algorithm:

1. Start.
2. Define a Patient class with attributes: patient_id, name, age, disease, doctor, and admitted (status flag).
3. Define a HospitalManagementSystem class that maintains a dictionary of patients and an auto-incrementing ID counter.
4. Implement add_patient(): creates a new Patient object, stores it in the dictionary, and increments the ID counter.
5. Implement discharge_patient(id): sets the patient's admitted flag to False if the ID exists.
6. Implement search_patient(id): looks up and displays a patient's details by ID.
7. Implement update_patient(id, ...): modifies the disease and/or doctor fields of an existing patient record.
8. Implement display_all_patients(): prints every record in the system, regardless of status.
9. Implement display_admitted_patients(): filters and prints only patients whose admitted flag is True.
10. In the main program, demonstrate the system by adding sample patients, displaying records, searching, updating, discharging, and re-displaying the results.
11. Stop.

Program (Anaconda / Python)

```
class Patient:
```

```
    def __init__(self, patient_id, name, age, disease, doctor):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.disease = disease
        self.doctor = doctor
        self.admitted = True

    def __str__(self):
        status = "Admitted" if self.admitted else "Discharged"
        return (f"ID: {self.patient_id} | Name: {self.name} | Age: {self.age} | "
                f"Disease: {self.disease} | Doctor: {self.doctor} | Status: {status}")
```

```
class HospitalManagementSystem:
```

```
    def __init__(self):
        self.patients = {}
        self.next_id = 1

    def add_patient(self, name, age, disease, doctor):
        patient = Patient(self.next_id, name, age, disease, doctor)
        self.patients[self.next_id] = patient
        print(f"Patient '{name}' admitted successfully with ID {self.next_id}.")
        self.next_id += 1

    def discharge_patient(self, patient_id):
        if patient_id in self.patients:
            self.patients[patient_id].admitted = False
            print(f"Patient ID {patient_id} discharged successfully.")
        else:
            print(f"No patient found with ID {patient_id}.")

    def search_patient(self, patient_id):
        patient = self.patients.get(patient_id)
        if patient:
            print("Patient Found:")
            print(patient)
        else:
```

```
print(f'No patient found with ID {patient_id}.')

def update_patient(self, patient_id, disease=None, doctor=None):
    patient = self.patients.get(patient_id)
    if patient:
        if disease:
            patient.disease = disease
        if doctor:
            patient.doctor = doctor
        print(f'Patient ID {patient_id} updated successfully.')
    else:
        print(f'No patient found with ID {patient_id}.')

def display_all_patients(self):
    if not self.patients:
        print("No patient records available.")
        return
    print("\n--- All Patient Records ---")
    for patient in self.patients.values():
        print(patient)

def display_admitted_patients(self):
    admitted = [p for p in self.patients.values() if p.admitted]
    print("\n--- Currently Admitted Patients ---")
    if not admitted:
        print("No patients currently admitted.")
    for patient in admitted:
        print(patient)

# ----- DEMONSTRATION -----
if __name__ == "__main__":
    hms = HospitalManagementSystem()

    print("===== HOSPITAL MANAGEMENT SYSTEM =====\n")

    # Adding patients
    hms.add_patient("Ramesh Kumar", 45, "Diabetes", "Dr. Anitha")
    hms.add_patient("Priya Sharma", 30, "Fracture", "Dr. Karthik")
    hms.add_patient("Suresh Babu", 60, "Hypertension", "Dr. Anitha")
```

```
# Display all patients
hms.display_all_patients()

# Search for a patient
print("\n--- Searching for Patient ID 2 ---")
hms.search_patient(2)

# Update patient record
print("\n--- Updating Patient ID 2 ---")
hms.update_patient(2, doctor="Dr. Meena")

# Discharge a patient
print("\n--- Discharging Patient ID 1 ---")
hms.discharge_patient(1)

# Display currently admitted patients
hms.display_admitted_patients()

# Final record of all patients
hms.display_all_patients()
```

Output:

===== HOSPITAL MANAGEMENT SYSTEM =====

Patient 'Ramesh Kumar' admitted successfully with ID 1.

Patient 'Priya Sharma' admitted successfully with ID 2.

Patient 'Suresh Babu' admitted successfully with ID 3.

--- All Patient Records ---

ID: 1 | Name: Ramesh Kumar | Age: 45 | Disease: Diabetes | Doctor: Dr. Anitha | Status: Admitted

ID: 2 | Name: Priya Sharma | Age: 30 | Disease: Fracture | Doctor: Dr. Karthik | Status: Admitted

ID: 3 | Name: Suresh Babu | Age: 60 | Disease: Hypertension | Doctor: Dr. Anitha | Status: Admitted

--- Searching for Patient ID 2 ---

Patient Found:

ID: 2 | Name: Priya Sharma | Age: 30 | Disease: Fracture | Doctor: Dr. Karthik | Status: Admitted

--- Updating Patient ID 2 ---

Patient ID 2 updated successfully.

--- Discharging Patient ID 1 ---

Patient ID 1 discharged successfully.

--- Currently Admitted Patients ---

ID: 2 | Name: Priya Sharma | Age: 30 | Disease: Fracture | Doctor: Dr. Meena | Status: Admitted

ID: 3 | Name: Suresh Babu | Age: 60 | Disease: Hypertension | Doctor: Dr. Anitha | Status: Admitted

--- All Patient Records ---

ID: 1 | Name: Ramesh Kumar | Age: 45 | Disease: Diabetes | Doctor: Dr. Anitha | Status: Discharged

ID: 2 | Name: Priya Sharma | Age: 30 | Disease: Fracture | Doctor: Dr. Meena | Status: Admitted

ID: 3 | Name: Suresh Babu | Age: 60 | Disease: Hypertension | Doctor: Dr. Anitha | Status: Admitted

EXPERIMENT 13: Implementation of Knowledge Representation Techniques

- a. Implement Semantic Network representation for a given knowledge domain (e.g., animal classification or vehicle types).
- b. Implement a Frame-based structure for a simple domain (e.g., student information or employee roles).

a. Implement Semantic Network representation for a given knowledge domain (e.g., animal classification or vehicle types).

Aim:

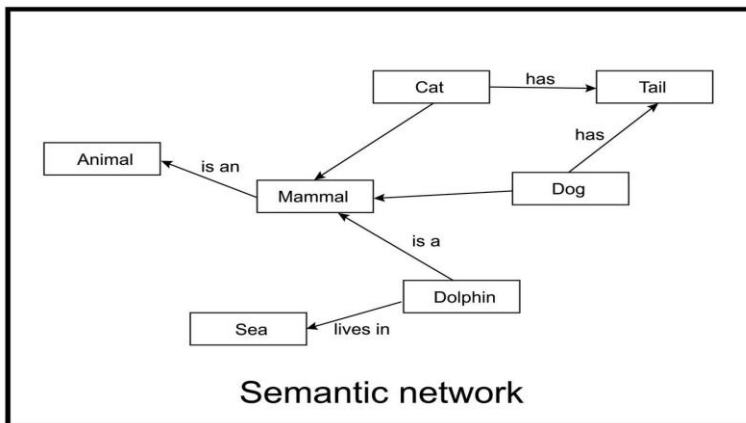
To implement and demonstrate **Semantic Network** and **Frame-Based Knowledge Representation** techniques for representing structured knowledge in Artificial Intelligence.

A. Semantic Network Representation**Description:**

A Semantic Network represents knowledge in the form of a **graph**, where nodes represent objects or concepts and edges represent relationships among them.

For example, in an animal classification domain:

- Animal → has → Life
- Bird → is-a → Animal
- Bird → has → Wings
- Sparrow → is-a → Bird
- Sparrow → can → Fly

**Algorithm:**

1. Start.
2. Define the concepts or entities in the selected knowledge domain.
3. Represent each concept as a node.
4. Define relationships such as is-a, has-a, can, etc.
5. Connect the nodes using the appropriate relationships.
6. Store the semantic network using a suitable data structure such as a dictionary or graph.
7. Accept a concept from the user if required.
8. Retrieve and display its relationships.

9. Stop.

Program

Semantic Network Representation

Domain: Animal Classification

```
class SemanticNetwork:
    def __init__(self):
        self.network = {}

    # Add a relationship between two concepts
    def add_relation(self, subject, relation, obj):
        if subject not in self.network:
            self.network[subject] = []
        self.network[subject].append((relation, obj))

    # Display the semantic network
    def display(self):
        print("\nSemantic Network:")
        print("-----")

        for subject in self.network:
            for relation, obj in self.network[subject]:
                print(subject, "--", relation, "-->", obj)

# Create Semantic Network
semantic_net = SemanticNetwork()

# Add animal classification knowledge
semantic_net.add_relation("Animal", "has", "Life")
semantic_net.add_relation("Bird", "is-a", "Animal")
semantic_net.add_relation("Bird", "has", "Wings")
semantic_net.add_relation("Bird", "has", "Feathers")

semantic_net.add_relation("Sparrow", "is-a", "Bird")
semantic_net.add_relation("Sparrow", "can", "Fly")

semantic_net.add_relation("Mammal", "is-a", "Animal")
semantic_net.add_relation("Mammal", "has", "Hair")

semantic_net.add_relation("Dog", "is-a", "Mammal")
```

```
semantic_net.add_relation("Dog", "can", "Bark")
```

```
semantic_net.add_relation("Cat", "is-a", "Mammal")
```

```
semantic_net.add_relation("Cat", "can", "Meow")
```

```
# Display network
```

```
semantic_net.display()
```

Output:

Semantic Network:

Animal -- has --> Life

Bird -- is-a --> Animal

Bird -- has --> Wings

Bird -- has --> Feathers

Sparrow -- is-a --> Bird

Sparrow -- can --> Fly

Mammal -- is-a --> Animal

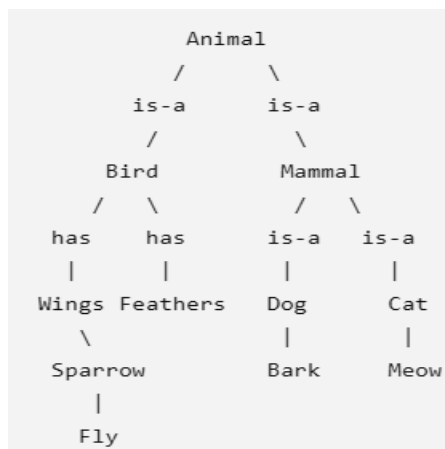
Mammal -- has --> Hair

Dog -- is-a --> Mammal

Dog -- can --> Bark

Cat -- is-a --> Mammal

Cat -- can --> Meow



B. Frame-Based Knowledge Representation

Description:

A **Frame** represents knowledge about an object or concept using a collection of **slots and values**. Each frame describes the properties or attributes of an entity.

For example, a Student frame may contain:

Slot	Value
Name	AAA
Roll Number	101
Branch	CSE
Year	III
CGPA	8.5

Frames can also support inheritance, where a specialized frame inherits properties from a more general frame.

Frame-Based Representation

Frame: stereotypical knowledge knowledge of some concept.

CLASS FRAME

Frame Name	Bird	
Properties	Color	unknown
	Eats	Worms
	#wings	2
	Flies	true
	Hungry	unknown
	Activity	unknown

INSTANCE FRAME

Frame Name	Tweety	
Properties	Color	yellow
	Eats	worms
	#wings	1
	Flies	False
	Hungry	unknown
	Activity	unknown

Algorithm:

1. Start.
2. Identify the entity to be represented, such as Student or Employee.
3. Create a frame for the entity.
4. Define the required slots or attributes.

5. Assign values to each slot.
6. Store the frame using a suitable data structure such as a dictionary or class.
7. Retrieve the required frame.
8. Display all slot-value pairs.
9. Stop.

Program:

Frame-Based Knowledge Representation

Domain: Student Information

class StudentFrame:

def __init__(self, name, roll_no, branch, year, cgpa):

Slots and their values

self.name = name

self.roll_no = roll_no

self.branch = branch

self.year = year

self.cgpa = cgpa

Method to display the frame

def display(self):

print("\n----- Student Frame -----")

print("Name :", self.name)

print("Roll Number :", self.roll_no)

print("Branch :", self.branch)

print("Year :", self.year)

print("CGPA :", self.cgpa)

Creating instances of Student Frame

student1 = StudentFrame("Aman", "101", "CSE", 3, 8.5)

student2 = StudentFrame("Rahul", "102", "AI & ML", 2, 8.8)

Display student information

student1.display()

student2.display()

Output:

----- Student Frame -----

Name : Aman

Roll Number : 101

Branch : CSE

Year : 3

CGPA : 8.5

----- Student Frame -----

Name : Rahul

Roll Number : 102

Branch : AI & ML

Year : 2

CGPA : 8.8

EXPERIMENT 14: Write a program to train and validate the following classifiers for given data

- Decision Tree
- Multi-layer Feed Forward Neural Network

A. Decision Tree Classifier**Aim**

To implement, train, and validate a **Decision Tree Classifier** on a given dataset and evaluate its classification performance.

Description

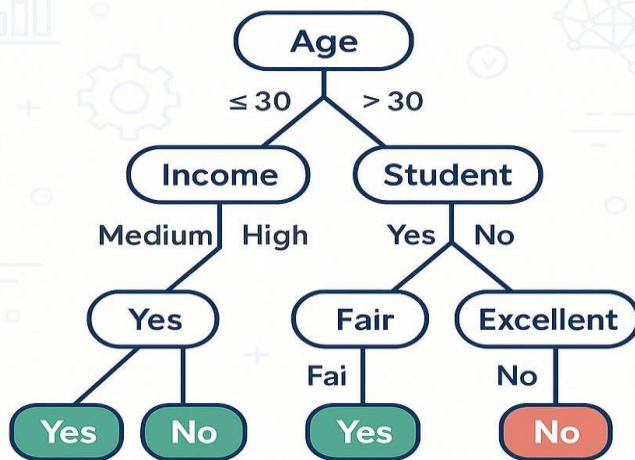
A **Decision Tree** is a supervised machine-learning algorithm used for classification and prediction. It represents decisions in the form of a tree structure.

The tree consists of:

- **Root Node:** Represents the complete dataset.
- **Decision/Internal Nodes:** Represent tests on features.
- **Branches:** Represent outcomes of the tests.
- **Leaf Nodes:** Represent predicted classes.

The dataset is divided into **training and testing sets**. The classifier is trained using the training data and validated using the testing data. Performance can be measured using accuracy, confusion matrix, precision, recall, and F1-score

DECISION TREE CLASSIFIER IN MACHINE LEARNING



Algorithm:

1. Start.
2. Import the required libraries.
3. Load the Iris dataset.
4. Separate the data into input features X and target values y.
5. Split the dataset into training data and testing data.
6. Create a Decision Tree Classifier.
7. Train the classifier using the training data.
8. Predict the classes for the testing data.
9. Compare the actual and predicted values.
10. Calculate the accuracy of the classifier.
11. Display the actual values, predicted values, and accuracy.
12. Stop.

Program:

```
# Decision Tree Classifier
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
# Load Iris dataset
```

```
data = load_iris()
```

```
# Input and output
```

```
X = data.data
```

```
y = data.target
```

```
# Split into training and testing data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create Decision Tree model
```

```
model = DecisionTreeClassifier()
```

```
# Train the model
```

```
model.fit(X_train, y_train)
```

```
# Test the model
```



```
y_pred = model.predict(X_test)

# Display results
print("Actual Values:")
print(y_test)

print("Predicted Values:")
print(y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Output:

Actual Values:

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 ...]

Predicted Values:

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 ...]

Accuracy: 1.0

The exact output can vary depending on the training/testing split and model settings.

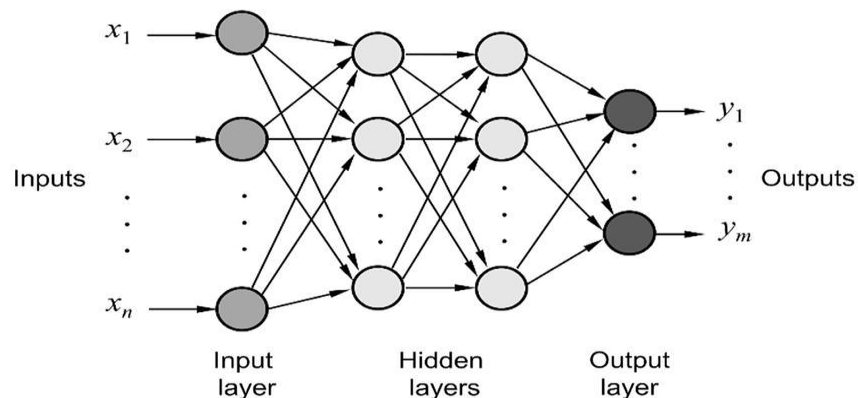
B. Multi-Layer Feed Forward Neural Network

Aim:

To implement, train, and validate a **Multi-Layer Feed Forward Neural Network (MLFFNN)** for classification of a given dataset and evaluate its performance.

Description:

A **Multi-Layer Feed Forward Neural Network** is an artificial neural network in which information moves in one direction—from the input layer through one or more hidden layers to the output layer.



A typical network consists of:

Input Layer → Hidden Layer(s) → Output Layer

Each neuron calculates a weighted sum of its inputs and applies an **activation function**. During training, the network adjusts its weights using **backpropagation** to reduce prediction error.

For a Python implementation, an MLPClassifier can be used to create a Multi-Layer Perceptron-based feed-forward neural network.

Algorithm:

1. Start.
2. Import the required libraries.
3. Load the Iris dataset.
4. Separate the input features X and target values y .
5. Split the dataset into training and testing data.
6. Create a Multi-Layer Feed Forward Neural Network using MLPClassifier.
7. Train the network using the training data.
8. Predict the classes for the testing data.
9. Calculate the accuracy of the model.
10. Display the actual values, predicted values, and accuracy.
11. Stop.

Program:

```
# Multi-Layer Feed Forward Neural Network
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()

X = data.data
y = data.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create neural network
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=42
)

# Train the network
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Display results
print("Actual Values:")
print(y_test)

print("Predicted Values:")
print(y_pred)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Output:

Actual Values:

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

Predicted Values:

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

Accuracy: 1.0

Explanation of Important Parameters

hidden_layer_sizes=(10,) means the neural network has one hidden layer containing 10 neurons.

max_iter=1000 specifies the maximum number of training iterations.

The network structure is:

4 Input Features → 10 Hidden Neurons → 3 Output Classes

The four inputs are sepal length, sepal width, petal length, and petal width, while the outputs represent the three Iris flower classes.